Resolve.

DOCUMENTATION LIBRARY / 05

Installation and deployment

Local runtime, identity and hosted preparation

Component setup, safe configuration examples, identity instructions and the deployment blueprint with recorded readiness limits.

COMPILATION DATE

12 November 2026

Source chapters retain their recorded dates and validation scope. Local, simulated and account-backed evidence are distinguished. This volume does not report a new production acceptance run.

Reading map

This volume brings together the maintained source documents listed below. Source chapters retain their recorded evidence dates and assumptions. Later corrections in the engineering log take precedence over older snapshots.

Chapter	Source in the project
Backend setup and configuration	backend/README.md
Frontend setup	frontend/README.md
Merchant sign-in	docs/MERCHANT_SIGN_IN.md
Buyer identity	docs/CUSTOMER_SIGN_IN.md
Deployment procedure	docs/deployment/deployment-guide.md
Recorded readiness assessment	docs/deployment/production-readiness-report.md

Backend setup and configuration

Source: backend/README.md

Resolve backend

For the source submission, the technical documentation PDF (../output/pdf/Resolve_Technical_Documentation.pdf) provides self-contained setup, architecture and API/SDK instructions. The source ZIP respects Git ignore rules and omits the repository's ignored `docs/` tree; some historical guide links below therefore refer outside that package.

Production preparation is documented in the Render deployment guide (../docs/deployment/deployment-guide.md) and readiness report (../docs/deployment/production-readiness-report.md). `GET /health` and the existing `POST /health` are liveness checks; `GET /ready` performs a read-only database probe and returns a sanitized 503 when storage is unavailable. API responses use private/no-store and nosniff headers. The public Sandbox PayPal webhook uses its configured app/webhook/merchant binding, not a browser merchant session. Hosted deployment and delivery remain unverified.

The real integration report (../docs/testing/production-integration-report.md) records the owner-authorized USD 1.00 official Sandbox order/capture/full-refund and independent persistence/UI checks. Other services remain explicitly blocked where credentials, consent or delivery authorization are missing. See the project-specific SDK guide (../docs/user-manual/sdk-integration-guide.md). Public customer refund responses now include `verification`; confirmed, pending and outstanding amounts are null until PayPal history is current. Consumers must handle null rather than treating missing proof as a confirmed zero.

The website's public `/docs` includes user and developer guides; the downloadable OpenAPI file is copied from the canonical contract, never from configured customer data. See Resolve guide (../docs/RESOLVE_GUIDE.md). After an API change, regenerate the canonical contract as usual and run `npm --prefix frontend run docs:export` from the repository root to refresh its website download.

Embedded order aftercare (../docs/SDK_AFTERCARE.md) adds bounded `POST /aftercare/issues`, `/aftercare/status` and `/aftercare/portal` for the SDK's customer-support workflow. Owners issue support keys with an explicit app source at `/platform/support-keys`. Existing commerce-only keys cannot use these new endpoints. Customer ownership must be checked by the merchant app server. SQLite 027/PostgreSQL 018 retain key grants and idempotent issue receipts. Polling status does not call providers; unverified/stale refund amounts are null, with the last check timestamp exposed.

Aftercare assurance (../docs/AFTERCARE_RELIABILITY.md) appends SQLite **028** and PostgreSQL **019**. New `/aftercare` actions expose safe refund journeys, context, return checks/labels, receipt lookup and resumable state updates to explicitly granted support keys. `/platform` adds return policies/inspections, refund tracing and versioned dispute holds. Buyer capabilities disclose active return terms before submission; an inactive policy does not quote a zero fee. The operations worker also refreshes source-scoped projections. App consumers poll this authenticated stream; it is not an outbound webhook subscription. Provider completion remains distinct from bank receipt. No Stripe/bank connector or label-purchase API is enabled by these changes.

The merchant operations guide (../docs/PLATFORM_EXPANSION.md) covers `/platform`: account signup/login/invites, roles and audit events, encrypted per-merchant connections, Zendesk imports/webhooks, overdue scans, dashboard, buyer offers, returns/replacements and dispute packages. CHG-048 introduced SQLite **026** and PostgreSQL **017**, followed by the aftercare migrations above. New migrations were validated on disposable databases; existing numbered migrations remain unchanged. Run `uv run python -m app.operations_worker --loop` from this directory to process queued imports and daily alerts. It reads the ordinary backend settings and never sends email or executes refunds.

Automatic resolution investigations use a durable evidence revision queue and reuse a selected linked resource from verified evidence only when the resource belongs to this case and its observation is newer than both five minutes and the configured freshness limit. A recent failed check is not a cache hit; stale evidence, incomplete history and foreign references remain blocked. Snapshot reuse is recorded as `investigation_read_cache_hit`; it grants no payment authority. The background worker still needs to be running. See CHG-060.

Investigations run in the separate `python -m app.investigation_worker` process. Groq triage and OpenRouter recommendations are optional and disabled by default; the default strict data policy fails closed for direct Groq. Finished and blocked results are private to full merchant credentials. Advisory review has no payment side effects. See configuration, API/SDK, offline tests and synthetic-only live evaluation (../docs/AI_INVESTIGATIONS.md). The investigation schema was introduced in SQLite 025 and PostgreSQL 015; merchant operations append later migrations.

FastAPI backend for recording customer cases, payment references, and refund commitments. Cases, refunds, and condition checks persist in SQLite by default or PostgreSQL when configured. Payment and refund details are submitted development data. An optional, disabled-by-default client can read PayPal Sandbox capture details and retain a scoped verification observation. Separate opt-ins enable approved Sandbox refunds and repeated partial execution after order-scoped reconciliation. See the execution contract (../docs/REFUND_EXECUTION.md) and workflow guide (../docs/PAYPAL_RECONCILIATION.md).

PostgreSQL now reuses a bounded connection pool (up to eight connections per store/process) instead of reconnecting for every transaction. Read transactions retain repeatable-read/read-only isolation; writes keep their transaction lock, rollback and approval rules. Browser read requests resolve the session once; write authorization still rechecks the current session. No cross-request session or financial-result cache is used. The API closes its pool on shutdown; callers using a standalone store can call `store.close()`. See performance and preview details (../docs/PERFORMANCE.md).

Run locally

From the `backend` directory:

```
uv sync
uv run uvicorn app.main:app --reload
```

Interactive API documentation: <http://127.0.0.1:8000/docs>.

Local environment files

Use `backend/.env` for database, PayPal, Google, email and evidence connectors; the `.env.example` lists those settings. In the prepared local file, missing optional credentials appear as commented `# NAME=` slots. Fill and uncomment the slots for an integration, then enable its switch. Existing values are preserved. Unused UUID and typed settings must stay commented rather than empty. Frontend URLs belong in `frontend/.env.local`; provider secrets stay on the backend.

AI settings deliberately read only process variables. Copy `ai-env.example.ps1` to `.env.ai.ps1` (the prepared workspace already has this private file), fill the provider keys, and set the desired feature switches. The default OpenRouter model is `openai/gpt-oss-120b` through `phala` (currently $0.15 input / $0.60 output per million tokens). For free use `xingchen-agi/xing4.0-29b-a4b:free` with the same route. Requests enforce a $0.74-per-million output price filter, strictly below the owner's $0.75 limit. Both models are in the default allowlist; model fallback is never automatic. From `backend`, load the file explicitly before starting a new API process:

```
. .\.env.ai.ps1
uv run uvicorn app.main:app --reload
```

Stop an existing API before restarting it on the same port. AI stays disabled in the template; the default `zdr_required` policy blocks direct Groq. Both Groq-based features require an explicit `minimized_standard_retention` policy selection to permit that provider's standard retention behavior.

Automatic investigations also need `uv run python -m app.investigation_worker --loop` in another configured terminal. That worker reads neither `.env` nor `.env.ai.ps1` automatically: load the PowerShell file and inject the same database URL as the API using its commented database slot. Optional Sandbox refresh also requires the corresponding existing Sandbox process variables. Do not start the worker with an unintended default SQLite database. Filling the files does not start workers or verify provider credentials. `.env`, `.env.ai.ps1` and frontend local settings are ignored by Git; the example files must contain no real secrets.

The merchant UI (../frontend/README.md) can also connect to this backend using the same merchant key. All workspace and buyer actions require the real FastAPI service; retired demo URLs do not provide simulated records. The customer

portal (../docs/CUSTOMER_PORTAL.md) uses expiring case links to show the customer-facing summary and conversation without merchant credentials.

Merchant access

Create a local merchant and token from the backend directory:

```
uv run python -m app.manage create-merchant --name "My development store"
```

The command prints a merchant ID, key ID, and `api_key` once. Save the token privately; the database retains only its SHA-256 hash. It is a random access token, not a password or PayPal credential. Merchants can also register individual owner accounts at `/account` when `RESOLVE_MERCHANT_SIGNUP_ENABLED=true` (default). An existing key-owned workspace can create its first owner at `/account?claim=1` after signing in with its full key. Do this to retain the existing workspace; creating a new workspace creates a separate tenant. Password sessions last eight hours. Owner invitations expire after two days and are shared manually.

Set `RESOLVE_PLATFORM_ENCRYPTION_KEY` before saving provider connections; use a Fernet key generated using the command in `.env.example`. The prepared ignored local `.env` has a generated key. Preserve it with database backups and share it with the operations/investigation/recovery workers; lost keys cannot decrypt saved connections. API responses show credential presence, never saved secrets.

In `/docs`, select **Authorize** and paste only the token. Requests use `Authorization: Bearer <api_key>`. `GET /auth/me` returns the authenticated merchant. All case, refund, condition, and reconciliation endpoints require authentication; `GET /health` and `POST /health` stay public. Missing, invalid, or revoked credentials return `401`. Another merchant's case returns `404`, just like a missing case.

Ownership comes from the token, never a request-body merchant ID. Capture and refund identifiers are scoped to a merchant: the same sample ID in another merchant's records neither exposes nor alters your balance. Shared capture history applies only within the owning merchant.

Additional local administrator commands:

```
uv run python -m app.manage issue-key --merchant-id <merchant_uuid>
uv run python -m app.manage revoke-key --key-id <key_uuid>
```

New keys do not revoke older keys automatically. Revocation blocks subsequent requests using that key. Tokens currently have no automatic expiry. These commands require trusted access to the database file; they are not exposed through HTTP. The merchant sign-in guide (../docs/MERCHANT_SIGN_IN.md) covers the browser session endpoints and optional PayPal identity. Individual staff accounts and owner/manager/finance/agent/viewer roles are implemented; see merchant operations (../docs/PLATFORM_EXPANSION.md).

Existing anonymous development records

Schema migration 2 preserves existing records under an unassigned legacy merchant with no access key. Creating a merchant does not automatically grant those records. To explicitly assign all unassigned sample records to your development merchant:

```
uv run python -m app.manage claim-legacy --merchant-id <merchant_uuid>
```

The assignment is atomic and refuses conflicting capture/refund IDs in the target merchant. Existing case IDs and evidence notes are preserved. Run this only when those old local records belong to that merchant. No automatic ownership inference or PayPal-account ownership verification is performed.

PayPal Sandbox capture reads

This integration is disabled by default and only targets `https://api-m.sandbox.paypal.com`. It exchanges sandbox app credentials for an OAuth access token and reads an existing capture; it cannot create payments or refunds.

First-time setup

1. Sign in to the PayPal Developer Dashboard (https://developer.paypal.com/dashboard/).

2. Under **Testing Tools** > **Sandbox Accounts**, use/create a business seller and

a personal buyer. These are separate from live PayPal accounts.

3. Under **Apps & Credentials**, select **Sandbox**, create a Merchant app tied

to that business account, and copy the client ID and client secret privately.

4. Use the sandbox seller's login at sandbox.paypal.com (https://sandbox.paypal.com/signin/).

Under the profile menu, open **Account Settings** > **Business information** and note **PayPal Merchant ID**. This is distinct from Resolve's UUID returned by `create-merchant` or `GET /auth/me`.

5. Copy the optional fields in `.env.example` to your ignored `backend/.env` and

fill them locally. Never commit or paste the client secret into chat.

6. Restart the backend. Make a sandbox purchase using your app and sandbox buyer

with the official checkout sample (https://developer.paypal.com/checkout/integrate). Its captured order response contains `purchase_units[].payments.captures[].id`. Create a Resolve case using that actual **capture ID** and original amount. A PayPal order ID is not a capture ID. `DEMO-*` sample IDs will not verify.

7. In Resolve's `/docs`, authorize with your local Resolve token and call

`POST /cases/{case_id}/paypal-capture-verifications`. Then inspect its saved history and the case's reconciliation. Use the same sandbox seller/app for checkout and verification.

Put the following fields in `backend/.env`, replacing placeholders locally. Uncomment the fields from `.env.example` and set the enabled flag to `true` only after filling them:

```
RESOLVE_PAYPAL_SANDBOX_ENABLED=true
RESOLVE_PAYPAL_RESOLVE_MERCHANT_ID=YOUR_RESOLVE_MERCHANT_UUID
RESOLVE_PAYPAL_SANDBOX_CLIENT_ID=YOUR_SANDBOX_APP_CLIENT_ID
RESOLVE_PAYPAL_SANDBOX_CLIENT_SECRET=YOUR_SANDBOX_APP_CLIENT_SECRET
RESOLVE_PAYPAL_SANDBOX_MERCHANT_ID=YOUR_SANDBOX_SELLER_MERCHANT_ID
```

Keep existing database configuration if you already have an `.env`. Sandbox login email/password, REST app credentials, and the local Resolve bearer token have different purposes; the client uses only the REST credentials for PayPal calls.

Setup references: PayPal getting started (https://developer.paypal.com/api/get-started/) and sandbox accounts (https://developer.paypal.com/sandbox-testing/accounts/). The merchant ID instructions (https://developer.paypal.com/docs/multiparty/seller-onboarding/build-onboarding/) also describe where to find the sandbox account's ID; partner onboarding from that document is not required by this first-party client.

The first-party sandbox app is bound to exactly one Resolve merchant UUID in configuration. Other merchants cannot borrow its credentials. This does not yet implement partner onboarding or per-merchant credential storage.

Verification endpoints

• `POST /cases/{case_id}/paypal-capture-verifications` reads the case's capture

and appends a verification observation. No request body or client-supplied URL is used. Authentication and case ownership are required.

• `GET /cases/{case_id}/paypal-capture-verifications` reads saved history for that

merchant/capture, including after credentials are disabled.

Verification requires exact capture ID, original gross amount, currency, and configured payee merchant ID. Unsupported currency, missing payee, malformed provider JSON, or unknown statuses cannot produce a successful verification. The official schema permits the payee field but does not guarantee it in every response; this version refuses incomplete data instead of guessing. An order-read fallback is future work.

Successful records distinguish `capture_details_verified=true` from `refund_history_verified=false` and `refund_eligibility_verified=false`. Capture status is reported as observed, including pending/refunded states. Verification does not establish that a refund is permitted, confirm return evidence, or import a complete refund history. Overall reconciliation/approval `verified_with_paypal` therefore remains false.

The record is persisted, appears in reconciliation, contributes to its state fingerprint, and invalidates existing approvals for that payment. The external read happens outside the database write transaction; the backend rechecks the case's payment before saving the result. Provider errors never create a successful observation or overwrite payment amounts.

`checked_at` is the server save time of a historical observation. Saved evidence does not expire automatically and must be refreshed before future execution.

Disabled/incomplete configuration returns `503`; a different configured Resolve merchant returns `403`; missing captures return `404`; amount/payee mismatches return `409`; invalid capture ID syntax returns `422`; malformed/denied provider reads return `502`; unavailable/rate-limited provider returns `503`; timeout returns `504`. Raw provider error bodies and secrets are not returned.

Real sandbox verification requires your own app credentials and a sandbox payment. The configured app's OAuth exchange was checked successfully against PayPal Sandbox on 2026-11-07. Subsequently its capture client read the owner's USD 1.00 test capture, matching ID, original amount, currency and configured seller, with observed status COMPLETED. This direct client check did not create a case or save a provider observation at that point. The authenticated case-route and full-refund checks subsequently passed, as recorded in the execution section below. Follow the manual test purchase walkthrough (../docs/SANDBOX_TEST_PURCHASE.md) to create a sandbox order, approve it as a buyer, capture it, and verify it in Resolve. The owner reported completing the manual purchase. The assistant read its capture but did not send any order-creation or capture-write request. Local simulation results establish caller behavior against fixtures, not a real PayPal account. See the simulation guide (../postman/README.md).

Local database

On startup, the initial versioned SQL migration creates `backend/data/resolve.sqlite3` if needed. Records survive reloads and restarts. Importing the app alone does not create the database.

The optional `RESOLVE_DATABASE_PATH` setting accepts a local file path, either in the environment or in `backend/.env`; see `.env.example`. Relative paths resolve from the backend directory, independently of the terminal working directory.

SQLite remains the local default. Set `RESOLVE_DATABASE_URL` in `.env` to select PostgreSQL for the API, merchant CLI and both workers. Use the migration guide (../docs/POSTGRES_MIGRATION.md) to copy an existing SQLite database into an empty target. The merchant CLI's explicit `--database` option continues to select a SQLite file.

Payments and refunds store integer cents; the API still uses decimal strings. SQLite writes start with `BEGIN IMMEDIATE`. PostgreSQL writes use a transaction lock shared by Resolve processes connected to the same schema. Both paths serialize financial state changes and give reconciliation a consistent read snapshot. Lock contention exceeding the wait timeout returns `503`.

Schema version 20 is recorded in SQLite's `user_version`. Startup applies the initial, merchant-ownership, refund-approval, capture-verification, durable execution/refund-evidence, provider-identity, case-event/condition-epoch, and refund-history/case-lifecycle, shared resolution workspace, customer portal, intake/triage, hosted follow-up, case ordering, and merchant browser authentication migrations atomically as needed. Version-1 records are preserved as described above. An unknown newer version fails startup instead of resetting it. The PayPal flow claim column is installed by SQLite migration 015 and PostgreSQL migration 004. Buyer Google sign-in and the email transport settings add SQLite 016 and PostgreSQL 005. Email notification flags are added by SQLite 017 and PostgreSQL 006. Evidence SDK records and check snapshots are added by SQLite 018 and PostgreSQL 007. SQLite 019 and PostgreSQL 008 add connected app

snapshots and scoped commerce credentials. SQLite 020 and PostgreSQL 009 preserve those snapshots in the merchant order catalog and allow order registration before a case exists. Put future schema changes in new migrations.

Old in-memory records are not automatically imported. Databases and `.env` files are ignored by Git. Stop the server before copying the file for a local backup. Keep the file on local disk; this is not a multi-host or network-filesystem setup.

API contract

Use the commerce integration (../docs/COMMERCE_INTEGRATION.md) to send versioned order, payment, product, and optional contact evidence from an existing merchant app. Generate SDK keys from the regular workspace's Settings screen after a recent access-key sign-in, or use the merchant CLI. It reuses the case's PayPal read path to report field-level differences. A commerce-scoped key can sync and refresh without access to contact reads, merchant sign-in, or payment execution.

See buyer sign-in and email (../docs/CUSTOMER_SIGN_IN.md) for Google sign-in, the default Resend transport, and the optional Nodemailer SMTP transport.

Use the Resolve Evidence SDK (../sdk/typescript/README.md) from a merchant's server with the FastAPI base URL and an existing merchant access key. It can open or reuse a case, link order/payment references, and record merchant evidence such as a refund approval. Refresh uses the configured PayPal Sandbox connection to read one USD capture and its supported order/refund records. The evidence summary reports the most recent check, preserves previously verified facts on failure, and identifies what could not be verified. Use the case timeline and bundle for support staff; the existing private buyer link shows status labels without exposing evidence or payment references.

See customer intake and case triage (../docs/CUSTOMER_WORKFLOW.md) for order invites, the public status timeline, scoped evidence uploads, inbox filters, and linking an existing payment to an intake case. The optional follow-up worker and account integrations (../docs/HOSTED_FOLLOWUP.md) add transactional emails, My cases, scheduled proof reads, and Zendesk ticket status. These connections are off unless explicitly configured.

- `GET /health` and `POST /health`: public liveness check.
- `GET /auth/me`: authenticated merchant identity.
- `POST /cases`: create a case; returns `201` with an ID and status `open`.
- `GET /cases`: owned case page with `limit`, `offset`, and optional `capture_id`.
- `GET /cases/{case_id}`: retrieve a case; returns `404` when no case matches.
- `PUT /cases/{case_id}/commitment`: revise a commitment using the current state fingerprint.
- `GET /cases/{case_id}/timeline`: recorded payment/case history, paginated and merchant-scoped.
- Invalid input returns `422`.

Case responses include server-assigned `merchant_id`. Condition-check responses include `recorded_by_merchant_id`; neither field can be supplied in its create request. These identify the authenticated merchant, not a verified PayPal account.

Merchant refund approvals

Approvals record merchant consent to an exact refund against the current local case state. They do not execute refunds or establish PayPal eligibility.

1. Review `GET /cases/{case_id}/reconciliation`, including its `state_fingerprint`.
2. Submit the exact recommended amount, capture, currency, and fingerprint below.
3. Retrieve the approval, or revoke it before a future executor uses it.

`POST /cases/{case_id}/refund-approvals` accepts:

```
{
  "request_id": "8fdf964d-a1b0-4870-978c-9ba28ed076c0",
  "capture_id": "DEMO-CAPTURE-001",
  "amount": "80.00",
  "currency": "USD",
  "state_fingerprint": "replace with the 64-character reconciliation fingerprint",
  "decision_note": "Approve the remaining refund after reviewing the return."
}
```

Use a new UUID for each new decision. Replace the fingerprint placeholder with the actual value. The server rechecks agreement, conditions, pending refunds, amount, currency, capture, and fingerprint in the transaction saving the approval. An ineligible case, stale state, or existing active approval returns `409`.

Endpoint	Behavior
`POST /cases/{id}/refund-approvals`	Save approval; `201` initially, `200` for an identical replay.
`GET /cases/{id}/refund-approvals`	List the case's approval records.
`GET /cases/{id}/refund-approvals/{approval_id}`	Retrieve the approval and saved reconciliation basis.
`POST /cases/{id}/refund-approvals/{approval_id}/revoke`	Revoke with `{"reason": "Changed my decision"}`.
`GET /cases/{id}/refund-approvals/{approval_id}/events`	Read approval, revocation, expiry, and invalidation events.

The server sets the approving merchant, approval ID, and 15-minute expiry. Only one approval is active per merchant/capture, including across cases. New refund observations, changed refund status, new condition checks, and successful PayPal capture observations invalidate active approvals for that capture within the same transaction. Identical refund replays and repeated same-status updates do not invalidate approvals.

Approval reads persist expiry events they discover. Expired, revoked, and invalidated records remain visible. Replaying their original request returns the terminal record without reactivating it. A fresh decision needs a new request UUID; a different body with an existing UUID returns `409`. Identical revocation retries are idempotent.

The approval stores an immutable local state snapshot and event history. It does not change refund totals. The bounded sandbox executor rechecks exact consent and provider state, reserves one durable full-refund action, and reuses its persisted PayPal request ID on recovery. Claimed approvals cannot be revoked to cancel an already submitted action. All approval routes require authentication and return `404` for foreign cases or approval IDs.

New case requests require a `payment` object. A `commitment` may be omitted or `null`, because a complaint can exist before a remedy has been proposed.

```
{
  "order_id": "ORDER-001",
  "complaint": "My refund is incomplete.",
  "payment": {
    "capture_id": "DEMO-CAPTURE-001",
    "captured_amount": "100.00",
    "currency": "USD"
  },
  "commitment": {
    "amount": "100.00",
    "currency": "USD",
    "source_message": "We will refund the full $100 after receiving your return.",
    "condition": "Return must be received",
    "status": "accepted"
  }
}
```

Validation rules

• This first version supports USD only; both currency fields default to USD.

• Monetary values use `Decimal`. Send decimal strings, such as `"100.00"`.

• Amounts must be finite, greater than zero, and have at most 12 significant

decimal digits, including at most two fractional digits.

• A refund commitment is the total agreed refund, not an additional refund. It

cannot exceed the submitted captured amount and must use the same currency.

• A commitment defaults to `proposed`; `accepted` records an agreement, not

authorization to execute a refund.

• An optional condition is recorded as text. Manual checks and evidence notes

can be recorded through the condition-check endpoints below.

• Required text is trimmed and cannot be blank. Unknown fields are rejected,

including within payment and commitment objects.

• Monetary values are serialized as JSON strings in responses.

These checks validate submitted data. Merchant authentication protects local records; explicit approval records and scoped sandbox capture verification are supported. Verified order history is scoped to one supported PayPal Sandbox capture; the application does not claim merchant-wide reporting coverage.

Sandbox refund execution and provider observations

Read the execution contract (../docs/REFUND_EXECUTION.md) for request bodies, state transitions and recovery. Set `RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED=true` locally only to opt into sandbox writes; capture reads must also be enabled. The default is false, and production hosts are not configurable.

By default execution supports the first full refund only. With `RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED=true`, refresh order history before approval, then approve an exact amount up to the current recommendation. Resolve re-reads history before executing and rejects stale approvals or gaps. PayPal's response remains the authority on actual acceptance.

Execution states are `reserved`, `submitting`, `unknown`, `pending`, `completed`, and `failed`. Inspect the returned status: creating an execution record is not proof that a refund completed. An unknown outcome retains a capture hold with `execution_pending` / `monitor_refund_execution` and zero recommendation, without inventing a completed/pending refund. Manual ledger writes are blocked during the hold. Recorded provider refund status must be refreshed from PayPal.

Retries reuse the same saved provider UUID, body, client identity and seller. The 60-second worker lease and 24-hour recovery POST limit prevent competing submissions and retries outside this version's supported window. A prior unknown outcome stays unknown when a later retry is rejected. `refresh` may receive a known `refund_id` to reconcile an unknown action by a provider read instead.

Provider refund observations are exposed separately and included in the latest reconciliation evidence. Their individual `refund_details_verified=true` does not assert complete refund history or general eligibility; overall verification remains false. Projection, approval invalidation, execution events and observation history commit together. A failure after a provider request retains recoverable state rather than reporting a fictitious failed refund.

On 2026-11-07, the existing USD 1.00 sandbox test completed through the actual authenticated routes: exact test approval, one refund POST, independent refund GET, capture status REFUNDED and reconciliation fulfilled. A temporary key/server were cleaned up, and the write flag was enabled only inside that test process. The test commitment was

explicitly synthetic; no customer conversation was invented.

Case workspace and recovery worker

The resolution workspace API (../docs/RESOLUTION_WORKSPACE.md) adds `/resolution-cases` for several independently verified promises in one case, including cases without a captured payment. It can link an existing refund case and use internal return evidence or carrier milestones as prerequisites for that refund. It exposes revisions, a private evidence timeline and a dispute evidence export. The guide lists the optional read-only provider connectors and request examples.

Use the workflow guide (../docs/PAYPAL_RECONCILIATION.md) for attachment uploads, close/reopen, the review queue, and webhook registration and processing.

See backend operations (../docs/BACKEND_OPERATIONS.md). A case can start without a commitment, then receive a proposed/accepted revision using the current reconciliation fingerprint. Different stale changes return `409`; an identical commitment is an idempotent no-op. Amount/currency remain constrained by the capture. Changed conditions start a new evidence epoch, preserving old checks without reusing them to satisfy the revised condition. Active executions block financial commitment changes.

The timeline combines durable case/manual-refund changes with existing approval, execution, condition and provider-observation history. It preserves source case IDs within the owner's shared capture and never fabricates legacy timestamps. Actor labels identify the merchant account, system, or unknown legacy actor; they do not establish a particular person's identity. Response pages are bounded; the current local timeline assembles a capture's recorded history before slicing.

```
uv run python -m app.worker --once
uv run python -m app.worker --loop --interval 30 --limit 20
```

The trusted local worker recovers only the configured merchant's existing reservations. It creates no agreement or approval, observes backoff/leases and the recovery window, and can refresh pending known refunds while writes are disabled. It is not installed as a system service or hosted scheduler.

Regenerate the reusable contract after API/model changes:

```
uv run python scripts/export_openapi.py
```

Output: Resolve OpenAPI (../postman/contracts/resolve.openapi.json). Exporting does not start a server or initialize the database. It exports public schema definitions, not local credentials or case records.

Refund tracking and reconciliation

The refund and condition submission routes in this section accept manual local development data. Provider observations/executions use their separate routes above. Recording a manual refund does not execute one. Accepted commitments and positive recommendations do not authorize a financial operation.

Endpoint	Behavior
`POST /cases/{id}/refunds`	Record a refund observation; `201` for a new refund, `200` for an identical replay.
`GET /cases/{id}/refunds`	List the payment's refund records, shared across cases referencing the same capture.
`PATCH /cases/{id}/refunds/{refund_id}`	Update a pending refund to `completed` or `failed`.
`POST /cases/{id}/condition-checks`	Record a manual `met` or `not_met` condition check with an evidence note.
`GET /cases/{id}/condition-checks`	Retrieve that case's condition-check history.
`GET /cases/{id}/reconciliation`	Calculate totals, outstanding commitment, and the next step from recorded data.

Record a refund

```
{
  "refund_id": "DEMO-REFUND-001",
  "capture_id": "DEMO-CAPTURE-001",
  "amount": "20.00",
  "currency": "USD",
  "status": "completed"
}
```

Refund statuses are `pending`, `completed`, and `failed`. Refund IDs are unique within the merchant's ledger. Identical replays return the existing record; a replay with different details returns `409`. Changes to status use the PATCH endpoint:

```
{"status": "completed"}
```

Pending refunds can become completed or failed. Terminal states cannot be changed; repeating the current status is allowed. Capture, amount, and currency are immutable. The record must reference the case's capture, and completed plus pending refunds cannot exceed the capture amount. Failed refunds consume no balance. Conflicting records return `409`; invalid field values return `422`. Missing cases or refunds return `404`.

The same merchant's cases sharing a capture must agree on its amount and currency. They see the same refund history, so opening another case does not reset that payment's balance.

Record a condition check

For the sample return condition, submit:

```
{
  "status": "met",
  "evidence_note": "Development example: warehouse return record RETURN-001 confirms receipt."
}
```

Checks record the authenticated merchant ID, are timestamped, and are preserved in order. Legacy checks retain an unknown actor rather than inventing one. The latest check determines the recorded condition state. A case without a condition returns `409` when a check is submitted. This is manual evidence recording, not independent verification.

Reconciliation rules

• Completed refunds reduce the outstanding commitment.

• Pending refunds reserve capture balance but do not count as completed remedies.

• Failed refunds consume no balance and do not fulfill commitments.

• `outstanding_commitment_amount` includes amounts still pending.

• `uncovered_commitment_amount` subtracts both completed and pending refunds from

the committed total and is floored at zero.

• `remaining_refundable_amount` subtracts completed and pending refunds from the

captured amount. It is based only on the local records, not PayPal eligibility.

• `recommended_refund_amount` is zero until an agreement is accepted, any condition

is recorded as met, and no refund is pending. It is a suggested amount to review, not an executable action or authorization.

• Outstanding amounts are `null` if there is no commitment.

• Reconciliation does not close the case or mutate its agreement status.

Situation	Reconciliation status	Next action
No commitment	`no_commitment`	`review_complaint`
Reserved, submitting, unknown or pending execution	`execution_pending`	`monitor_refund_execution`
Terminally failed first full-refund execution	`execution_failed`	`review_failed_execution`
Proposed commitment	`awaiting_acceptance`	`confirm_agreement`
Accepted commitment, a refund pending	`refund_pending`	`monitor_pending_refund`
Accepted commitment fully refunded, no pending refund	`fulfilled`	`none`
Accepted commitment, outstanding amount, condition not met or unchecked	`awaiting_condition`	`verify_condition`
Accepted commitment, outstanding amount, condition met or absent, no pending refund	`refund_required`	`request_merchant_approval`

With an accepted $100 commitment, a recorded satisfied condition, and $20 completed, the outstanding and recommended amounts are $80. If a further $80 is pending, the outstanding amount stays $80 but the recommendation is $0. If that pending refund fails, the recommendation becomes $80 again. A completed refund can fulfill an accepted commitment even if a prior condition was not checked; it has already been paid according to the submitted records.

Suggested remedies

`GET /resolution-cases/{id}/remedies` builds remedy options in code (no model call).

Supported options:

• **Refund after the return:** the refundable balance minus the restocking fee

from the enabled return policy. The return milestone is `inspection_passed` when the policy requires inspection.

• **Partial refund (the buyer keeps the item):**

`RESOLVE_REMEDY_PARTIAL_REFUND_PERCENT` of the refundable balance; default 20, range 1-90.

Refundable balance: captured minus completed and pending refunds, from the existing reconciliation.

When no refund option is offered: no payment is linked, a dispute or provider hold applies, a refund promise is already open, or nothing is left to refund.

Not offered: store credit and replacement. The promise models need an issued Shopify credit transaction or a shipment tracker before they can be recorded.

AI ranking: `POST /resolution-cases/{id}/ai/rank-remedies` classifies the issue and the buyer's preference, ranks the listed options with a one-line reason each, and drafts a buyer message.

• **Input:** the masked conversation. It reuses the extraction switch, data

policy, cache, token cap and daily budget.

• **Guard:** an option, amount, number or unoffered remedy (replacement, store

credit and similar) outside the list returns code order and a template.

• **Advisory:** it never records or publishes anything.

Creating promises: `POST /resolution-cases/{id}/remedies/promises` takes `{option_id, expected_amount}`.

• **Re-check:** it recomputes the option and returns `409` if the balance

changed or the option is no longer valid.

• **What it records:** the promises (a return promise plus a refund that

requires it, or one refund), and it returns an `offer_choice`.

• **One refund promise per capture:** a capture holds one cumulative refund

promise, so one option is recorded per case.

Publishing: the existing `POST /platform/cases/{id}/offers` endpoint, called separately by the merchant. It now rejects a refund choice above the current refundable balance, or one on a held payment, with `409`.

AI cache: the extraction, explanation and ranking caches store only the model's raw output. Every cache hit is parsed and validated again against the current case.

Tests

```
uv run python -m pytest
```

The default tests exercise the local FastAPI application with PayPal disabled. The separate HTTP simulation tests (../postman/README.md) are skipped unless explicitly enabled behind the local proxy. Neither test suite contacts real PayPal or an AI provider. Each test uses its own temporary database; tests do not clear the development database. Persistence checks reopen the file in new app instances and a separate Python process. Concurrency checks use independent stores sharing one local file.

Frontend setup

Source: frontend/README.md

Resolve merchant UI

For the production audit, run `npx playwright test --config playwright.audit.config.ts` from this directory. It builds a disposable source copy, uses synthetic storage, checks page widths at desktop/tablet/mobile sizes and captures real application screenshots under `docs/user-manual/screenshots`. Optional `RESOLVE_AUDIT_SCENARIO` values `paypal-email` and `google` require their corresponding local Postman TLS simulations. Never use this fixture against an owner's configured database. See the deployment guide (../docs/deployment/deployment-guide.md) for Render; no hosted production URL has been verified by this audit.

Public documentation lives at `/docs`, linked from the landing page and workspace. Ten guides are authored in `content/docs.json`; static article pages and search use that source. `npm run docs:export` refreshes `../docs/RESOLVE_GUIDE.md` and the public guide/OpenAPI downloads; `npm run docs:check` detects stale exports. Normal and preview build commands export these assets first. Documentation requires no backend or sign-in. `npx playwright test -c playwright.docs.config.ts` checks the compiled pages, search, copy/download actions and small-screen navigation using an isolated source copy with no API server.

Aftercare assurance (SDK 0.7) adds refund tracing, reusable support context and return policy/label workflows. See the workflow guide (../docs/AFTERCARE_RELIABILITY.md).

Operations -> Connections -> Customer support inside your store now generates an explicit support integration key for an app source. The merchant app can submit order issues, read safe status and issue private buyer links through the SDK gateway. Order-only key generation remains available. See the SDK aftercare guide (../docs/SDK_AFTERCARE.md) for the server-side ownership check.

Use `/account` to create a workspace, sign in as an individual, or accept an owner-created invitation. `/workspace/operations` includes the dashboard, missed promises, support imports, staff access, provider connections, SDK keys and audit history. Existing workspace owners can sign in with a full key, then open `/account?claim=1`. Case details include offers, guided returns and reviewed dispute packages; private buyer links show safe offers and return requests. See merchant operations setup (../docs/PLATFORM_EXPANSION.md). `npx playwright test -c playwright.platform.config.ts` runs the isolated desktop and mobile workflow against synthetic SQLite data and no provider accounts.

Each merchant case includes a private **Automatic investigation** panel with findings, source IDs, missing/contradictory evidence, freshness and the proposed next step or abstention. Approve, reject or request evidence using the existing merchant session. This advisory review is separate from refund approval and submission. Run the separate backend investigation worker, including with inference disabled, to produce reviewable results. See the investigation guide (../docs/AI_INVESTIGATIONS.md) and `npx playwright test -c playwright.investigations.config.ts` for the isolated browser test. `tests/investigations.gateway.mjs` checks the actual session/API gateway over local HTTP when a browser cannot be launched.

The regular merchant workspace includes connected orders (../docs/COMMERCE_INTEGRATION.md#show-all-orders-in-the-merchant-dashboard) at `/workspace/orders`. Apps publish to the same merchant through the Evidence SDK. Merchants can browse and search orders and open a case from its details. Generate and revoke commerce keys in **Settings -> SDK keys**; copy the server configuration with the key and the web app's `/api/sdk` URL.

The Next.js app opens with a public landing page. Sign in to the real backend for the merchant inbox, case detail, promises, evidence, review queue and printable buyer receipt. The customer portal (http://127.0.0.1:3000/customer) needs a private case/intake link. `/demo` redirects to sign-in; legacy sample query strings never create fake records or successful financial outcomes.

For normal local use, run the optimized preview from this directory:

```
npm run build:preview
npm run preview
```

Keep the backend running separately on port 8000. The preview uses its own ignored `.next-preview` build folder and serves port 3000; stop another frontend on 3000 before starting it. Rebuild and restart after source/configuration changes. Use `npm run dev` while editing when automatic reload is needed. These modes should not run on the same port. Performance details (../docs/PERFORMANCE.md).

Cases show their summary before payment details finish loading. Evidence timeline, Zendesk, investigations, offers, returns and dispute tools load when opened. The timeline is reused within the selected case, and switching main case tabs preserves the customer heading draft. Customer messages still have an explicit refresh. Investigations stop polling when their section is collapsed. Refund controls still need freshly loaded payment/reconciliation data.

From this directory, for development:

```
npm ci
npm run dev
```

Open <http://127.0.0.1:3000>, create/sign into your merchant account and use the configured backend. Missing provider credentials produce an unavailable/error state. Product code has no mock fallback. Controlled test doubles remain confined to opt-in automated tests and are never evidence of real provider success.

To work with actual local case records:

1. In a separate terminal, run `uv run uvicorn app.main:app --reload` from

`backend`.

2. Select **Merchant sign in** and enter an existing private Resolve merchant

key. If you need a key, use the backend account setup (../backend/README.md).

3. Select **New case** to create a standalone case, link an existing refund

case, or record a PayPal payment. Record the promise in the case. A linked refund promise must match the accepted commitment on its payment case.

4. Check provider evidence, then review an exact refund amount. Approval and

submission are separate. Connected refund submissions use the existing opt-in PayPal Sandbox path.

If the backend uses another port or service, copy `.env.example` to `.env.local` and set `RESOLVE_BACKEND_URL`. The browser gets an opaque HttpOnly token for a durable backend session; it never receives the access key back. See merchant sign-in (../docs/MERCHANT_SIGN_IN.md) for PayPal linking and production start commands.

To share a case, open **Customer portal** in the case desk, choose a heading, and create the link. Customers see progress and can exchange messages without a merchant key. Links expire after seven days and can be replaced or revoked. For links that work outside this machine, configure an HTTPS public origin. See the customer portal guide (../docs/CUSTOMER_PORTAL.md).

To let a buyer start a new case, choose **Invite customer** and enter the order reference. The link opens the intake form, then the case timeline, evidence uploads, and conversation. Choose **Manage case** to set assignment, due date, and whether the case is waiting on the buyer. The inbox can filter by workflow or unread messages; **Customer portal** contains the buyer's files and replies. See the full walkthrough (../docs/CUSTOMER_WORKFLOW.md). Use the actual issued intake link; `?intake=1` does not bypass authorization.

The optional hosted follow-up setup (../docs/HOSTED_FOLLOWUP.md) enables email invites, the **My cases** email sign-in screen, and an optional merchant Zendesk card. Copyable private invites work through the backend without email.

`npm run check` checks TypeScript; `npm run build` builds production assets. To run browser tests, install Playwright's Chromium with `npx playwright install chromium`, or set `PLAYWRIGHT_CHROMIUM_EXECUTABLE` to your local Chrome

executable, then run `npm run test:e2e`. These tests use ports 3100 and 8011, temporary SQLite, and a synthetic merchant. They do not use the configured development database.

To include the payment and PayPal login browser tests, run from the repository root:

```
$env:RESOLVE_UI_PAYPAL_SIMULATION = '1'
$env:RESOLVE_UI_PAYPAL_LOGIN = '1'
$env:RESOLVE_UI_EMAIL_FIXTURE = '1'
postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift "npm --prefix frontend run
test:e2e" --output ndjson
```

The simulated provider tests use fake credentials. The Playwright readiness checks for local services bypass the provider proxy. For product and implementation scope, see the frontend plan (../docs/FRONTEND_PLAN.md) and engineering log (../docs/ENGINEERING_LOG.md).

Merge integration checks include the platform workflows above and the incoming promise-health/settings screens. Run the compiled isolated checks with `RESOLVE_UI_PRODUCTION_TEST=1` and `npx playwright test -c playwright.investigations.config.ts` from this directory (PowerShell: set the variable with `$env:RESOLVE_UI_PRODUCTION_TEST='1'`). The API's canonical export and local schema references are checked by backend `tests/test_openapi_contract.py`.

Merchant sign-in

Source: docs/MERCHANT_SIGN_IN.md

Resolve landing page and merchant sign-in

Individual accounts are also available at `/account`, with self-service workspace creation and owner-created staff invitations. `/workspace/operations` manages the team and connections. To retain an existing key-owned workspace, sign in with its full key then open `/account?claim=1` to create its first owner account. Staff roles apply to API writes as well as UI actions. See the merchant operations guide (PLATFORM_EXPANSION.md) for permissions and limitations.

Route	What it opens
`/`	Public landing page with merchant, buyer and documentation entry points
`/demo`	Retired sample URL; redirects to `/login` and does not create sample records
`/login`	Merchant sign-in with a Resolve access key or linked PayPal account
`/workspace`	Signed-in merchant inbox
`/workspace/settings`	Link or unlink PayPal, review session details, and sign out
`/customer`, `/customer/account`	Private buyer link and separate My cases sign-in

An administrator creates or issues a merchant access key through `app.manage`. Use that key at `/login` the first time. A successful sign-in sets an HttpOnly browser session; the API stores only a hash of the session token. A session lasts eight hours. Sign-out revokes it, and revoking an access key also invalidates sessions issued using that key. Existing integrations can continue using their provisioned API bearer keys.

Enable Log in with PayPal

1. In the PayPal Developer Dashboard (https://developer.paypal.com/log-in/build),

open the chosen Sandbox REST app and enable **Log in with PayPal**. Add the exact return URL `https://YOUR-RESOLVE-HOST/api/auth/paypal/callback`. Use the same origin when opening Resolve and when configuring the app. PayPal also asks for Privacy Policy and User Agreement URLs. Its docs permit example URLs for Sandbox testing; use your real published policies for live review.

2. Set these variables on the **backend**:

```
RESOLVE_MERCHANT_WEB_ORIGIN=https://YOUR-RESOLVE-HOST
RESOLVE_PAYPAL_LOGIN_ENABLED=true
RESOLVE_PAYPAL_LOGIN_ENVIRONMENT=sandbox
RESOLVE_PAYPAL_LOGIN_CLIENT_ID=YOUR_SANDBOX_LOGIN_APP_CLIENT_ID
RESOLVE_PAYPAL_LOGIN_CLIENT_SECRET=YOUR_SANDBOX_LOGIN_APP_CLIENT_SECRET
```

Set `RESOLVE_MERCHANT_WEB_ORIGIN` to that same origin on the **frontend**. Set `RESOLVE_CUSTOMER_PUBLIC_ORIGIN` on both if the buyer portal has a different host or if you enable customer email (HOSTED_FOLLOWUP.md). Keep login credentials and the database URL on the backend. Refund API settings use separate variables and are unaffected.

3. Sign in with the merchant access key, open **Settings -> Link PayPal**, and

complete PayPal consent in the same browser. After that, sign out and use **Log in with PayPal**. Linking or unlinking requires a key session created in the last 30 minutes; unlinking revokes active PayPal sessions for that workspace.

Resolve requests the `openid` identity scope. It does not store PayPal access tokens or refresh tokens and does not match accounts by email. The PayPal account can open only the Resolve workspace to which it was explicitly linked. A browser session cannot enable refund writes or replace PayPal's separate seller permissions

(https://developer.paypal.com/platforms/seller-onboarding/onboarding-checklist/). PayPal requires review for live Log in with PayPal (https://developer.paypal.com/log-in/build#go-live); do not switch to the live environment until the app and exact return URL are approved and tested.

Deployment notes

• For a directly exposed Next.js service, build it and run

`npm run start:hosted`. The regular `npm start` binds to loopback for a reverse proxy on the same host.

• Set frontend `RESOLVE_BACKEND_URL` to the FastAPI origin. Plain HTTP is

allowed on loopback; use HTTPS for an external host, or set `RESOLVE_BACKEND_ALLOW_PRIVATE_HTTP=true` only on a trusted private network. Preserve the external `Host` header when proxying browser routes.

• Use FastAPI `GET /health` as a deployment liveness check. The original

POST endpoint still works. Backend startup applies pending migrations (currently through SQLite 020 or PostgreSQL 009).

• Restrict public backend ingress to the existing PayPal webhook and

appropriate health routes if the frontend can reach FastAPI privately. Configure access logs to omit query strings on the PayPal callback.

• The repository verifies OAuth against a local HTTPS simulation. Register

and test an actual PayPal app and HTTPS callback before relying on it for a deployment.

Buyer identity

Source: docs/CUSTOMER_SIGN_IN.md

Buyer sign-in and email

Buyers can open their private case link without an account. To return later, they can use **My cases** at `/customer/account` with Google or a one-time email link. Neither option grants access to the merchant workspace.

Enable Google for My cases

1. In Google Auth Platform (https://console.cloud.google.com/auth/clients),

create a **Web application** OAuth client. Set the consent screen and add a test user if the project's audience is restricted. Register `https://YOUR-FRONTEND-HOST/api/customer-account/google/callback` as an **authorized redirect URI**. Use your actual buyer frontend URL.

2. Add these values to the FastAPI environment:

```
RESOLVE_CUSTOMER_PUBLIC_ORIGIN=https://YOUR-FRONTEND-HOST
RESOLVE_CUSTOMER_GOOGLE_ENABLED=true
RESOLVE_GOOGLE_CLIENT_ID=YOUR_WEB_APPLICATION_CLIENT_ID
RESOLVE_GOOGLE_CLIENT_SECRET=YOUR_WEB_APPLICATION_CLIENT_SECRET
```

Set the same `RESOLVE_CUSTOMER_PUBLIC_ORIGIN` in the frontend environment; credentials stay on the backend. For Google-only local testing, use the loopback origin registered with the exact callback. Email delivery still requires a public HTTPS origin.

3. Restart both apps. From the merchant inbox, create an order invite using

the buyer email already verified for that order. Leave **Email this invite from Resolve** unchecked if you are sharing the link yourself. After the buyer submits the issue, they can choose **Sign in with Google** using the same email and open their case. This works with email delivery disabled.

Resolve verifies Google's signed ID token and uses the matching email to find already assigned cases. Verified Gmail and Google Workspace addresses with a matching hosted domain are supported. For other addresses, use the private case link or verify the mailbox with the email sign-in option. See Google's web OAuth setup (https://developers.google.com/identity/protocols/oauth2/web-server) and ID token guidance (https://developers.google.com/identity/gsi/web/guides/verify-google-id-token). Configure hosted request logs to omit OAuth callback query strings. The buyer button uses a Google-provided brand asset (https://developers.google.com/identity/branding-guidelines).

Enable buyer email

Use **Resend** for the simplest FastAPI deployment. It sends through the existing Python follow-up worker with no Node mail runtime. Set `RESOLVE_CUSTOMER_EMAIL_TRANSPORT=resend` (the default) and follow the hosted follow-up setup (HOSTED_FOLLOWUP.md) for the sender, API key, and worker.

Choose SMTP only if you already have a relay or prefer to use one. The existing follow-up worker invokes a small Nodemailer script for each email; there is no separate HTTP service or queue to deploy. Install Node.js and run `npm --prefix backend/mailer ci --omit=dev` on the machine running the worker. Then set these backend and worker variables:

```
RESOLVE_CUSTOMER_EMAIL_ENABLED=true
RESOLVE_CUSTOMER_EMAIL_TRANSPORT=smtp
RESOLVE_CUSTOMER_EMAIL_MERCHANT_ID=YOUR_RESOLVE_MERCHANT_UUID
RESOLVE_CUSTOMER_PUBLIC_ORIGIN=https://YOUR-FRONTEND-HOST
RESOLVE_CUSTOMER_SIGNING_KEY=YOUR_STABLE_RANDOM_SECRET
RESOLVE_SMTP_HOST=smtp.example.com
RESOLVE_SMTP_PORT=587
RESOLVE_SMTP_SECURE=false
```

```
RESOLVE_SMTP_USER=YOUR_SMTP_USERNAME
RESOLVE_SMTP_PASSWORD=YOUR_SMTP_PASSWORD
RESOLVE_SMTP_FROM=updates@example.com
```

Port 587 with `RESOLVE_SMTP_SECURE=false` requires STARTTLS. For a provider using implicit TLS, usually on port 465, set `RESOLVE_SMTP_SECURE=true`. Use an authorized sender address for the chosen SMTP account. Nodemailer SMTP reference (https://nodemailer.com/smtp)

Run `uv run python -m app.followup_worker --loop` from `backend` with the same PostgreSQL connection and email settings as the API. Test with your own address first. If you are switching transports, drain existing mail jobs first: a job with a previous attempt stays assigned to its original transport. For SMTP, only a definite temporary rejection is retried. An interruption after submission may mean the server accepted the message; the worker stops retrying that job. Acceptance means the provider took the message, not that it reached the inbox.

Deployment procedure

Source: docs/deployment/deployment-guide.md

Resolve deployment guide

Prepared 11 November 2026. Target: Render. **Deployment status: not deployed by this audit. No production URL or API URL has been verified.** Do not substitute a localhost address for a production URL.

Recommended architecture

Use the existing Next.js Node service as the public HTTPS website, a FastAPI Python web service as the authenticated API, managed PostgreSQL, and four Python background workers. Keep all services in one region. The browser calls same-origin Next.js gateways; only those gateways call the API. SDK servers can use `/api/sdk`. This preserves the current architecture, session/Origin rules and durable jobs.

`render.yaml` defines six paid application services and one database, with automatic deployment disabled. Review current Render prices and resource sizes in your account before applying it. The chosen smallest paid plans are starting points, not measured production capacity. Render terminates HTTPS. The API is public for signed PayPal webhooks, authenticated SDK/server usage and health checks; database external access is denied by `ipAllowList: []`. The API has no permissive CORS middleware because browser calls use the website origin.

Attachments and PDF labels currently live in the database. No S3/R2 bucket is required. Back up this data and enforce operational storage limits. Moving attachments to object storage would be a separate implementation change.

Account and source prerequisites

1. Sign in to Render and connect the Git provider containing this repository. The audit could not access the Render browser host and found no Render deployment credentials. Account-backed resource creation was not attempted.

2. Review the complete working-tree diff, including changes that predate this audit. Authorize a commit/push, or publish the reviewed source yourself. Render builds a repository revision; uncommitted local changes cannot be deployed by pretending they are already in Git. This task has made no commit or push.

3. Enable billing if Render requires it and review the recurring charge for all six services and PostgreSQL. An account, repository permission and billing authorization are required before Apply.

4. Create a Blueprint from `render.yaml`. Keep auto-deploy off while configuring and validating the release. Use a new database first; do not point startup at the owner's current hosted database to experiment with migrations.

Required environment values

Set secrets in Render's environment controls. Never use `NEXT_PUBLIC_` for service credentials and never paste real values into this guide.

Variable	Set on	Value / rule
`RESOLVE_DATABASE_URL`	API and every worker	Blueprint internal PostgreSQL connection string. Use provider-supplied TLS settings for external databases. Never expose to frontend.

Variable	Set on	Value / rule
`RESOLVE_PLATFORM_ENCRYPTION_KEY`	Shared `resolve-production` environment group	Blueprint generates one base64-encoded 256-bit key, accepted by Fernet, and shares it with API/workers. Back it up privately. For an existing database, replace it with that

Variable	Set on	Value / rule
		database's existing key before saving/reading connections.
RESOLVE_MERCHANT_WEB_ORIGIN	Frontend, API, all workers	Exact final HTTPS origin, no path/query/fragment. Example shape: https://your-resolve-web.onrender.com. This is an example, not an assigned URL.
RESOLVE_CUSTOMER_PUBLIC_ORIGIN	Frontend, API, workers	Usually the same HTTPS origin; use a separate buyer host only after domain/routing checks.
RESOLVE_BACKEND_URL	Frontend only	Exact final API HTTPS origin. Example shape: https://your-resolve-api.onrender.com. No trailing resource path.
RESOLVE_MERCHANT_SIGNUP_ENABLED	API	Defaults true. Set false to close public workspace signup after intended onboarding.

For a manually managed deployment, generate the encryption key locally from backend:

```
uv run python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

This prints a new secret for you to store privately; do not record it in logs/screenshots. Never replace an existing key while encrypted provider connections still depend on it.

Render ignores sync: false inside environment groups. The Blueprint therefore prompts for origins directly on the frontend/API and generates the encryption key once in the group. Before enabling email/OAuth, also add the final public origins to the shared environment group so all workers receive them. Do not regenerate the group key during a redeploy.

All optional provider writes, email, Google/PayPal sign-in, automatic reads and AI start disabled in the Blueprint. Per-workspace encrypted connections can enable Sandbox operations separately, so review those records before starting workers against imported data. The root inventory lists every settings variable by name; backend/.env.example and backend/ai-env.example.ps1 explain optional combinations. AI settings come from process environment, not the ordinary dotenv loader.

The local owner-authorized USD 1.00 official PayPal Sandbox workflow is now verified through actual creation, buyer approval, capture, full refund, database and UI read-back. It is not a Render verification or permission to enable live payments. Integration status and exact missing configuration names (../testing/production-integration-report.md) separate that result from identity callbacks, delivery, carrier/helpdesk/store accounts and AI calls that remain blocked. Retain the default-disabled deployment settings until the intended account-specific acceptance run is prepared.

Build and start

The Blueprint uses locked dependencies and native runtimes; Docker is unnecessary. Docker Desktop's Linux engine was unavailable during this audit, so no container build is claimed.

Service	Working directory	Build	Start
Frontend	frontend	npm ci && npm run build	npm run start:hosted -- --port $PORT
API	backend	pip install uv==0.10.8 && uv sync --locked --no-dev	uv run --no-sync uvicorn app.main:app --host 0.0.0.0 --port $PORT --no-access-log

Service	Working directory	Build	Start
Operations worker	`backend`	Same Python build	`uv run --no-sync python -m app.operations_worker --loop`
Investigations worker	`backend`	Same Python build	`uv run --no-sync python -m app.investigation_worker --loop`
Follow-up worker	`backend`	Same Python build	`uv run --no-sync python -m app.followup_worker --loop`
Refund recovery worker	`backend`	Same Python build	`uv run --no-sync python -m app.worker --loop`

`$PORT` above is expanded by Render's Linux shell, not a literal Windows variable. Python is pinned to 3.12.12 and Node to major 22 in the Blueprint. Confirm runtime availability during Render validation. Build exports public guides and OpenAPI downloads before compiling Next.js. Build commands must run with the complete repository available because docs export reads `postman/contracts` outside `frontend`.

The follow-up worker defaults to Resend and needs no Node mailer. SMTP mode additionally requires Node plus `npm ci --prefix backend/mailer` in that worker's runtime; the default Blueprint does not enable SMTP. Configure and validate that runtime separately before selecting SMTP.

Database and migrations

API startup and workers apply the existing checksum-tracked migrations. SQLite has 028; PostgreSQL has 019. No schema migration was added for this audit. First deploy the API against a new empty PostgreSQL database, inspect startup and `/ready`, then start workers. Use managed backups/PITR before any later release.

For existing data, follow `docs/POSTGRES_MIGRATION.md`: use a read-only source snapshot, import into an empty disposable PostgreSQL schema, verify balances/ownership/counts, then schedule the real cutover. Do not run an unreviewed importer against the owner's configured storage. Never rewrite numbered migration SQL or stored checksums.

Domains, callbacks and connectivity

1. Record the actual Render-generated website and API URLs in your private release record. Set the exact origins above, then redeploy both services.

2. If using a custom domain, add it in Render, configure its supplied DNS records and wait for certificate verification. Update exact origins and restart all services after changing hosts.

3. Keep frontend-to-API connectivity on HTTPS. `RESOLVE_BACKEND_ALLOW_PRIVATE_HTTP` is only for a deliberately private network; do not enable it to make an insecure public URL work.

4. Register optional merchant PayPal callback `/api/auth/paypal/callback` on the website origin, buyer Google callback `/api/customer-account/google/callback` on the buyer origin, and signed Zendesk callback `/webhooks/zendesk/MERCHANT_UUID` on the frontend. Register PayPal payment webhooks against the backend route from its contract. Never enable optional login merely because an environment field exists.

Release verification

Use a synthetic workspace and a separate Sandbox account for provider checks. Keep real customer data out of recording artifacts.

- API `/health` must return 200 (liveness); `/ready` must return 200 only when database access works. Test a controlled database outage in staging: `/ready` must return 503 without connection details.
- Open every route in `docs/testing/application-inventory.md`; inspect desktop, tablet and mobile. Download both public documentation resources.
- Create workspace, sign out/in, check Secure/HttpOnly/SameSite cookies over HTTPS, test wrong Origin, expired/revoked credentials, denied staff writes, and tenant isolation.
- Create a case/promise, revise it, add evidence, share/revoke a buyer link, exchange synthetic messages, accept an offer, test return/label workflow, export reviewed evidence, sync and search an order, then refresh to verify persistence.
- Check worker logs/queue age and confirm only expected synthetic tasks run. Email requires a verified sender and a real delivery test. OAuth requires provider-registered callbacks and actual account consent. Test a full separate approval/submission/recovery flow only in PayPal Sandbox after explicit write enablement.
- Run the screenshot suite against the deployed release using an appropriate synthetic fixture/hosted configuration; never run the local fixture seeder against production customer storage. Replace local manual screenshots only with verified hosted screenshots and update their manifest.
- Configure uptime checks on `/ready`, service error/latency alerts, PostgreSQL connection/storage alerts, queue-age alerts, and tested backup restoration. Scrub tokens, URL fragments, provider responses, customer evidence and contacts from external logs.

Rollback

1. Disable public traffic or serve a maintenance response when financial state is uncertain. Pause workers before rollback; preserve queued jobs, leases, provider request IDs and uncertain outcomes.

2. Redeploy the last verified Git revision for frontend, API and all workers using Render's deployment controls. Keep environment secrets and encryption key unchanged. Verify database compatibility before starting older code.

3. Do not delete migration history or run a destructive down-migration. This audit adds no migration; future releases may need a forward repair. For a destructive database incident, restore a managed backup to a NEW database and reconcile subsequent events/provider state before switching.

4. Run `/ready`, authentication, owned-case reads, SDK connectivity and worker checks. Reconcile pending/uncertain refunds from provider evidence before allowing another execution. An application rollback cannot reverse an already submitted refund.

Authoritative references

- Render Blueprint specification (https://render.com/docs/blueprint-spec)
- Render background workers (https://render.com/docs/background-workers)
- Render deployment behavior (https://render.com/docs/deploys)
- Next.js self hosting (https://nextjs.org/docs/app/guides/self-hosting)
- FastAPI deployment (https://fastapi.tiangolo.com/deployment/docker/)

Provider-side Blueprint validation, resource provisioning, hosted callback checks, production load tests and rollback drills remain unperformed until account/source access is available.

Recorded readiness assessment

Source: docs/deployment/production-readiness-report.md

Production readiness assessment

Assessment date: 11 November 2026. **Status: NOT approved for production acceptance; local fixes and the authorized real Sandbox flow are verified, hosted rollout remains blocked.** This is not a penetration-test certification, availability guarantee or authorization to move live money. Product demo/fake success modes have been retired.

Verified implementation

Area	Evidence / finding
Secrets	Settings use SecretStr; local dotenv/database files are excluded by component ignores. Provider credentials are encrypted at rest; access/session tokens are hashed. No configured secrets/customer evidence were exported into the new documentation. Render secret values are absent from the Blueprint.
Merchant authentication	Staff scrypt passwords, durable eight-hour sessions, revocation and DB-backed authentication throttling; legacy key and optional linked PayPal login remain separate. Credential forms use POST and hydration guards.
Buyer isolation	Expiring/revocable capabilities and separate customer cookies; buyer requests cannot access merchant APIs. Private links are credentials and must be protected.
Authorization	Backend ownership/role checks; restricted SDK grants and app-source binding. Tests cover cross-tenant denial, role changes and guarded financial actions.
Financial safeguards	Decimal/cents precision, duplicate prevention, shared capture balances, pending/failed refunds, current acceptance/condition gates, approval expiry and uncertain-outcome recovery are covered by backend/simulation checks. The financial implementation remains Sandbox only.
Browser/API boundary	Same-origin gateways enforce exact Origin and body limits. No wildcard CORS enabled. Cookies become Secure on configured HTTPS; hosted proxy/cookie behavior still requires account-backed validation.
Input/files	Pydantic schemas, length/shape restrictions, bounded request/provider bodies, scoped attachments and saved PDF-label controls. Credential validation errors are private and do not reflect submitted credentials. Malware scanning and enterprise content retention are not implemented.
HTTP protection	Private API no-store, MIME protection, stricter buyer referrer/robots controls; frontend MIME/referrer/frame headers. Full CSP rollout is not claimed.
Database	Checksum-tracked immutable migrations; transactions, bounded connection pool and independent readiness. Disposable PostgreSQL checks passed after starting a fresh audit cluster; owner's hosted storage was not changed.
Provider webhook	Public PayPal webhook crash repaired; signature verification precedes persistence. Replay/conflict/reversal behavior passed local TLS simulation. Endpoint uses the configured global app/merchant binding; arbitrary per-workspace PayPal webhook routing is not implemented.
Dependencies	npm audits for frontend (including dev), SMTP mailer production, SDK and separate checkout store reported zero known vulnerabilities; pip-audit of 36 installed backend packages also reported none. Results are point-in-time advisory checks, not proof against unknown vulnerabilities.
Build/runtime	Compiled isolated Next.js pages and real disposable FastAPI routes were exercised. Development reload is absent from deployment commands. Render schema validation passes. Docker daemon unavailable; no container build claimed.
Performance	Lazy case tabs, timeline reuse, retained drafts and late-response rejection pass. Bounded triage batching/grouped complaint queries reduce the measured workload from 428 to 220 SQL statements/20 requests. Four local stages, 1/10/50/100 users, completed 3,220 requests with zero errors. P95 at 100 users was 1.70s. No production/soak capacity claim.
Real integration	Explicitly authorized official PayPal Sandbox order, buyer approval, capture and full refund independently verified; one persisted refund, zero remaining, actual SDK and merchant/buyer UI values match. Other account-backed services are blocked, not promoted to VERIFIED by mocks. See ../testing/production-integration-report.md.

Area	Evidence / finding
Truthful provider state	Public customer amounts are nullable with explicit verification; stale/unverified history is unavailable rather than a confirmed zero. Completion refreshes history read-only; failures never fabricate success or resubmit a payment.

Release blockers and material limitations

1. **No Render deployment/account access:** browser access host unavailable, no deployment credentials. Render billing/repository authorization and reviewed source publication remain required. Production URL, API URL, hosted screenshots and deployed user journeys cannot be verified yet.

2. **No production database/backup acceptance:** new local PostgreSQL is verified; managed instance provisioning, TLS/network policy, PITR, retention, capacity and restore drill must be tested in Render. Do not experiment on the owner's connected database.

3. **Optional integrations are not hosted-verified:** PayPal/Google identity consent/callbacks, signed delivery, Resend/SMTP sender/delivery, Zendesk/Shopify/EasyPost scopes and AI data policy need authorized account-backed checks. The fresh official Sandbox transaction verifies only its documented local scope, not production or every provider action. Exact missing names/present-but-unverified credentials are listed in the integration report.

4. **Account lifecycle limits:** no MFA, email-verification requirement, password reset/recovery, cross-workspace staff membership or self-service provider OAuth installation. Decide whether these meet the intended public audience before opening unrestricted signup. An administrator must own access recovery.

5. **Abuse/operations controls:** auth throttles exist, but service-wide edge WAF/rate limiting, quotas, alert delivery and incident response are not provisioned. Configure edge and service monitoring before broad public access. Do not trust arbitrary forwarded client-IP headers to bypass per-peer throttles.

6. **Financial operations limits:** no live PayPal refunds, bank receipt verification, automatic dispute submission or purchased return labels. Never describe the demonstration as handling live funds. Pause recovery/follow-up workers and reconcile ambiguous provider state when rolling back.

7. **QA scope:** Chromium at four widths, six targeted Firefox checks, all 175 operations' negative probes, 172 operations' successful pytest observations and bounded local load do not prove Safari, full Firefox, accessibility compliance, physical-device behavior, production high concurrency or every possible state permutation. Three positive callback/unlink ledger gaps remain explicit. The test report identifies all unperformed scopes.

8. **Storage/privacy:** database-backed attachments increase DB/backups and need retention/access processes. Exported dispute snapshots can contain private evidence. Define legal/business retention and support policies for the actual operating organization before collecting customers' records.

Operating requirements

Assign an owner for uptime checks on `/ready`, 5xx/latency alerts, database connections/storage, worker crashes/queue age, secrets rotation, backups and incident response. Store the Fernet key with private backups. Use a synthetic staging workspace and separate Sandbox integration for release tests. Restrict signup until onboarding/support and abuse controls are ready. Keep tokens, private fragments and provider/customer payloads out of external telemetry.

Use `deployment-guide.md` for release and rollback. This audit supplies the files and verified local evidence; Render's account-side validation and acceptance are the next release gate.

Backend configuration template

Source: backend/.env.example. Examples only; no private settings are read.

```
# Optional local SQLite file. Relative paths are resolved from backend/.
# RESOLVE_DATABASE_PATH=data/resolve.sqlite3
# Optional: use PostgreSQL for FastAPI, the merchant CLI, and both workers.
# Keep the real URL in backend/.env only. Use TLS options from your provider.
# RESOLVE_DATABASE_URL=postgresql://ROLE:PASSWORD@HOST/DATABASE?sslmode=require

# Individual merchant accounts and encrypted per-workspace provider connections.
RESOLVE_MERCHANT_SIGNUP_ENABLED=true
# Generate once with: uv run python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
# Keep the same private key in API/worker processes and backups. Do not regenerate on restart.
# RESOLVE_PLATFORM_ENCRYPTION_KEY=

# Browser sign-in (optional PayPal identity flow; separate from payment credentials).
# Set the same hosted merchant origin on the frontend.
# RESOLVE_MERCHANT_WEB_ORIGIN=https://resolve.example.com
RESOLVE_PAYPAL_LOGIN_ENABLED=false
RESOLVE_PAYPAL_LOGIN_ENVIRONMENT=sandbox
# RESOLVE_PAYPAL_LOGIN_CLIENT_ID=
# RESOLVE_PAYPAL_LOGIN_CLIENT_SECRET=

# PayPal reads remain disabled until you configure all fields locally.
RESOLVE_PAYPAL_SANDBOX_ENABLED=false
# Separate opt-in for approved sandbox refunds:
RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED=false
# Also require fresh, verified order history and allow partial/multiple refunds:
RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED=false
# Webhook ID issued by the same Sandbox REST app when registering the HTTPS URL:
# RESOLVE_PAYPAL_SANDBOX_WEBHOOK_ID=
# Resolve merchant UUID from create-merchant / GET /auth/me:
# RESOLVE_PAYPAL_RESOLVE_MERCHANT_ID=
# Sandbox REST app credentials; these are not your sandbox login password:
# RESOLVE_PAYPAL_SANDBOX_CLIENT_ID=
# RESOLVE_PAYPAL_SANDBOX_CLIENT_SECRET=
# PayPal sandbox seller's merchant ID; distinct from the Resolve UUID:
# RESOLVE_PAYPAL_SANDBOX_MERCHANT_ID=

# Optional read-only resolution evidence connectors, restricted to one Resolve merchant:
# RESOLVE_RESOLUTION_CONNECTOR_MERCHANT_ID=
# RESOLVE_SHOPIFY_SHOP_DOMAIN=your-store.myshopify.com
# RESOLVE_SHOPIFY_ADMIN_TOKEN=
# RESOLVE_EASYPOST_API_KEY=
RESOLVE_AUTO_PROVIDER_REFRESH_ENABLED=false

# Optional transactional email and My cases. Use a public HTTPS customer origin.
RESOLVE_CUSTOMER_EMAIL_ENABLED=false
# Select resend or smtp. SMTP uses Nodemailer in the follow-up worker.
RESOLVE_CUSTOMER_EMAIL_TRANSPORT=resend
# RESOLVE_CUSTOMER_EMAIL_MERCHANT_ID=
# RESOLVE_CUSTOMER_PUBLIC_ORIGIN=https://support.example.com
# RESOLVE_CUSTOMER_SIGNING_KEY=
# RESOLVE_RESEND_API_KEY=
# RESOLVE_RESEND_FROM=updates@example.com
# RESOLVE_SMTP_HOST=smtp.example.com
# RESOLVE_SMTP_PORT=587
# False requires STARTTLS; true uses implicit TLS (typically port 465).
# RESOLVE_SMTP_SECURE=false
# RESOLVE_SMTP_USER=updates@example.com
# RESOLVE_SMTP_PASSWORD=
# RESOLVE_SMTP_FROM=updates@example.com

# Optional Google sign-in for buyers, independent of email delivery.
RESOLVE_CUSTOMER_GOOGLE_ENABLED=false
# RESOLVE_GOOGLE_CLIENT_ID=
# RESOLVE_GOOGLE_CLIENT_SECRET=
# Register /api/customer-account/google/callback on the buyer frontend origin.

# Optional read-only Zendesk connection, scoped to one merchant:
```

```
# RESOLVE_HELPDESK_MERCHANT_ID=
# RESOLVE_ZENDESK_SUBDOMAIN=
# RESOLVE_ZENDESK_OAUTH_TOKEN=

# All AI variables have an editable PowerShell template: ai-env.example.ps1.
# Copy it to .env.ai.ps1, fill values, and run: . .\.env.ai.ps1 before starting
# the backend. AI values added to this dotenv file are deliberately not loaded.
# Optional AI promise extraction (proposals only; never records promises or moves money).
# Like app.inference, its settings are read from the process environment only,
# never from this file. Set them in the server's secret store or shell:
#   RESOLVE_EXTRACTION_ENABLED=true            (default false)
#   RESOLVE_AI_GROQ_KEY=...                    (shared with app.inference)
#   RESOLVE_AI_DATA_POLICY=minimized_standard_retention
#                                              (default blocks; see docs/AI_INVESTIGATIONS.md)
#   RESOLVE_EXTRACTION_MODEL=openai/gpt-oss-120b
#   RESOLVE_EXTRACTION_TIMEOUT_SECONDS=20
#   RESOLVE_EXTRACTION_DAILY_TOKEN_LIMIT=100000  (per merchant per UTC day, this process only)
```

AI process configuration template

Source: backend/ai-env.example.ps1. Examples only; no private settings are read.

```
# Copy to .env.ai.ps1, fill values, then run: . .\.env.ai.ps1
# This explicitly sets process variables; AI settings do not read backend/.env.
# Keep .env.ai.ps1 private (it is ignored by Git). Restart the API after loading.

# Shared provider credentials. Fill these locally; never add real keys here.
$env:RESOLVE_AI_GROQ_KEY = ''
$env:RESOLVE_AI_OPENROUTER_KEY = ''

# Promise suggestions from support conversations. Enable after filling Groq key.
$env:RESOLVE_EXTRACTION_ENABLED = 'false'
$env:RESOLVE_EXTRACTION_MODEL = 'openai/gpt-oss-120b'
$env:RESOLVE_EXTRACTION_TIMEOUT_SECONDS = '20'
$env:RESOLVE_EXTRACTION_DAILY_TOKEN_LIMIT = '100000'

# Automatic case investigations (requires the separate investigation worker).
$env:RESOLVE_AI_ENABLED = 'false'
$env:RESOLVE_AI_GROQ_MODEL = 'openai/gpt-oss-20b'
# GPT-OSS 120B through Phala: currently $0.60 / million output tokens.
# Optional free model: xingchen-agi/xing4.0-29b-a4b:free (same Phala route).
# Requests also cap provider output pricing at $0.74 / million tokens.
$env:RESOLVE_AI_OPENROUTER_MODEL = 'openai/gpt-oss-120b'
$env:RESOLVE_AI_APPROVED_GROQ_MODELS = '["openai/gpt-oss-20b","openai/gpt-oss-120b"]'
$env:RESOLVE_AI_APPROVED_OPENROUTER_MODELS = '["xingchen-agi/xing4.0-29b-a4b:free","openai/gpt-oss-120b"]'
$env:RESOLVE_AI_OPENROUTER_ROUTES = '["phala"]'

# Default blocks direct Groq. To permit its standard retention policy, explicitly
# change to minimized_standard_retention. Filling a key alone does not opt in.
$env:RESOLVE_AI_DATA_POLICY = 'zdr_required'
$env:RESOLVE_AI_ALLOW_SANDBOX_REFRESH = 'false'
$env:RESOLVE_AI_TIMEOUT_SECONDS = '12'
$env:RESOLVE_AI_RETRIES = '1'
$env:RESOLVE_AI_MAX_TOKENS = '1800'
$env:RESOLVE_AI_FRESHNESS_SECONDS = '3600'

# Investigation worker only: it does not load backend/.env. Before running it,
# uncomment and fill the same database URL used by the API. Otherwise it selects
# the default local SQLite database. Inject any needed Sandbox settings too if
# enabling RESOLVE_AI_ALLOW_SANDBOX_REFRESH.
# $env:RESOLVE_DATABASE_URL = ''

# From backend/, after filling and loading this file:
# uv run uvicorn app.main:app --reload
# In a separate terminal with the same environment and database configuration:
# uv run python -m app.investigation_worker --loop
```

Frontend configuration template

Source: frontend/.env.example. Examples only; no private settings are read.

```
# Default for a co-located FastAPI backend. For separate services, use its
# HTTPS origin or opt in to a trusted private-network HTTP endpoint.
RESOLVE_BACKEND_URL=http://127.0.0.1:8000
# RESOLVE_BACKEND_ALLOW_PRIVATE_HTTP=true
# Exact public merchant origin, matching backend settings and PayPal callback.
# RESOLVE_MERCHANT_WEB_ORIGIN=https://resolve.example.com
# Optional: set the same exact HTTPS customer origin as the backend.
# RESOLVE_CUSTOMER_PUBLIC_ORIGIN=https://support.example.com
```

Deployment blueprint

Source: render.yaml. Examples only; no private settings are read.

```
# Paid resources. Review plans and connect the repository in Render before Apply.
# No credentials or existing customer database are included.
envVarGroups:
  - name: resolve-production
    envVars:
      - key: RESOLVE_PLATFORM_ENCRYPTION_KEY
        generateValue: true
      - key: RESOLVE_PAYPAL_SANDBOX_ENABLED
        value: "false"
      - key: RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED
        value: "false"
      - key: RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED
        value: "false"
      - key: RESOLVE_CUSTOMER_EMAIL_ENABLED
        value: "false"
      - key: RESOLVE_CUSTOMER_GOOGLE_ENABLED
        value: "false"
      - key: RESOLVE_PAYPAL_LOGIN_ENABLED
        value: "false"
      - key: RESOLVE_AUTO_PROVIDER_REFRESH_ENABLED
        value: "false"
      - key: RESOLVE_AI_ENABLED
        value: "false"
      - key: RESOLVE_EXTRACTION_ENABLED
        value: "false"
services:
  - type: web
    name: resolve-web
    runtime: node
    region: oregon
    plan: starter
    rootDir: frontend
    buildCommand: npm ci && npm run build
    startCommand: npm run start:hosted -- --port $PORT
    healthCheckPath: /
    autoDeployTrigger: "off"
    envVars:
      - key: NODE_VERSION
        value: "22"
      - key: NEXT_TELEMETRY_DISABLED
        value: "1"
      - key: RESOLVE_BACKEND_URL
        sync: false
      - key: RESOLVE_MERCHANT_WEB_ORIGIN
        sync: false
      - key: RESOLVE_CUSTOMER_PUBLIC_ORIGIN
        sync: false
  - type: web
    name: resolve-api
    runtime: python
    region: oregon
    plan: starter
    rootDir: backend
    buildCommand: pip install uv==0.10.8 && uv sync --locked --no-dev
    startCommand: uv run --no-sync uvicorn app.main:app --host 0.0.0.0 --port $PORT --no-access-log
    healthCheckPath: /ready
    autoDeployTrigger: "off"
    envVars:
      - fromGroup: resolve-production
      - key: RESOLVE_MERCHANT_WEB_ORIGIN
        sync: false
      - key: RESOLVE_CUSTOMER_PUBLIC_ORIGIN
        sync: false
      - key: PYTHON_VERSION
        value: "3.12.12"
      - key: RESOLVE_DATABASE_URL
```

```
        fromDatabase:
          name: resolve-db
          property: connectionString
  - type: worker
    name: resolve-operations
    runtime: python
    region: oregon
    plan: starter
    rootDir: backend
    buildCommand: pip install uv==0.10.8 && uv sync --locked --no-dev
    startCommand: uv run --no-sync python -m app.operations_worker --loop
    autoDeployTrigger: "off"
    envVars:
      - fromGroup: resolve-production
      - key: PYTHON_VERSION
        value: "3.12.12"
      - key: RESOLVE_DATABASE_URL
        fromDatabase:
          name: resolve-db
          property: connectionString
  - type: worker
    name: resolve-investigations
    runtime: python
    region: oregon
    plan: starter
    rootDir: backend
    buildCommand: pip install uv==0.10.8 && uv sync --locked --no-dev
    startCommand: uv run --no-sync python -m app.investigation_worker --loop
    autoDeployTrigger: "off"
    envVars:
      - fromGroup: resolve-production
      - key: PYTHON_VERSION
        value: "3.12.12"
      - key: RESOLVE_DATABASE_URL
        fromDatabase:
          name: resolve-db
          property: connectionString
  - type: worker
    name: resolve-followup
    runtime: python
    region: oregon
    plan: starter
    rootDir: backend
    buildCommand: pip install uv==0.10.8 && uv sync --locked --no-dev
    startCommand: uv run --no-sync python -m app.followup_worker --loop
    autoDeployTrigger: "off"
    envVars:
      - fromGroup: resolve-production
      - key: PYTHON_VERSION
        value: "3.12.12"
      - key: RESOLVE_DATABASE_URL
        fromDatabase:
          name: resolve-db
          property: connectionString
  - type: worker
    name: resolve-recovery
    runtime: python
    region: oregon
    plan: starter
    rootDir: backend
    buildCommand: pip install uv==0.10.8 && uv sync --locked --no-dev
    startCommand: uv run --no-sync python -m app.worker --loop
    autoDeployTrigger: "off"
    envVars:
      - fromGroup: resolve-production
      - key: PYTHON_VERSION
        value: "3.12.12"
      - key: RESOLVE_DATABASE_URL
        fromDatabase:
          name: resolve-db
          property: connectionString
databases:
```

```
  - name: resolve-db
    region: oregon
    plan: basic-256mb
    databaseName: resolve
    user: resolve_app
    ipAllowList: []
```